The MCP Trilogy: Notes on the Model Context Protocol and an AI-Powered Newsletter Pipeline

> Study notes based on the trailer for **The MCP Trilogy** by Nitish (*CampusX*), with added explanations, architecture details, code examples, and practical takeaways.

Topic Model Context Protocol | **Level** Beginner to Intermediate | **Type** Learning Notes

Table of Contents

1. Overview
2. The MCP Trilogy Series Structure
3. Part 1 — The Why: The Problem MCP Solves
4. Part 2 — The What: MCP Architecture
5. Part 3 — The How: Building and Connecting MCP Servers
6. Case Study: The AI-Powered Newsletter Project
7. Why "Minimal Coding" Works
8. Additional Insights and Best Practices
9. Limitations and Things to Watch Out For
10. Ideas for Your Own MCP Projects
11. Glossary
12. Resources

1. Overview

The video is a trailer for a three-part mini-series called **The MCP Trilogy**, focused on the **Model Context Protocol (MCP)**. The host, Nitish from *CampusX*, shows how he used MCP to automate a complex, real-world task: producing a weekly AI newsletter from research to final HTML email (0:00–3:35).

What is MCP, in one sentence? MCP is an open standard, introduced by Anthropic in late 2024, that defines a common way for AI applications to connect to external tools, data sources, and services.

A popular analogy: **MCP is like a USB-C port for AI.** Before USB-C, every device needed its own cable. Before MCP, every AI app needed its own custom integration for every tool. MCP gives everyone one shared plug.

2. The MCP Trilogy Series Structure

Part	Title	Focus	Timestamp
1	**The Why**	Origin of MCP and the problems it addresses	1:42
2	**The What**	Technical architecture: servers, clients, and hosts	1:51
3	**The How**	Practical coding demos and real-world implementations	2:09

This structure is a good way to learn any new technology: understand the motivation first, then the design, then the implementation.

---

3. Part 1 — The Why: The Problem MCP Solves

3.1 LLMs are isolated by default

A large language model on its own only knows what was in its training data and what you paste into the chat. It cannot, by itself:

- Read your files or databases
- Check today's news or a GitHub repository
- Send an email, create a ticket, or run a query

To be useful in real workflows, an AI needs **context** (data) and **capabilities** (actions).

3.2 The M × N integration problem

Before MCP, connecting AI apps to tools looked like this:

```
M AI applications  ×  N tools  =  M × N custom integrations
```

If 5 AI apps each want to talk to 10 tools, that is **50 separate integrations**, each with its own authentication, data format, and maintenance burden.

With MCP:

```
M AI applications  +  N MCP servers  =  M + N implementations
```

Each tool is wrapped **once** as an MCP server, and each AI app implements the MCP client **once**. Any client can then talk to any server. The same 5 apps and 10 tools now need only **15** implementations.

3.3 Problems with the "old way"

- **Fragmentation:** every vendor had its own plugin or function-calling format.
- **Duplicated effort:** the same GitHub integration rebuilt for every AI product.
- **Vendor lock-in:** integrations built for one model did not carry over to another.

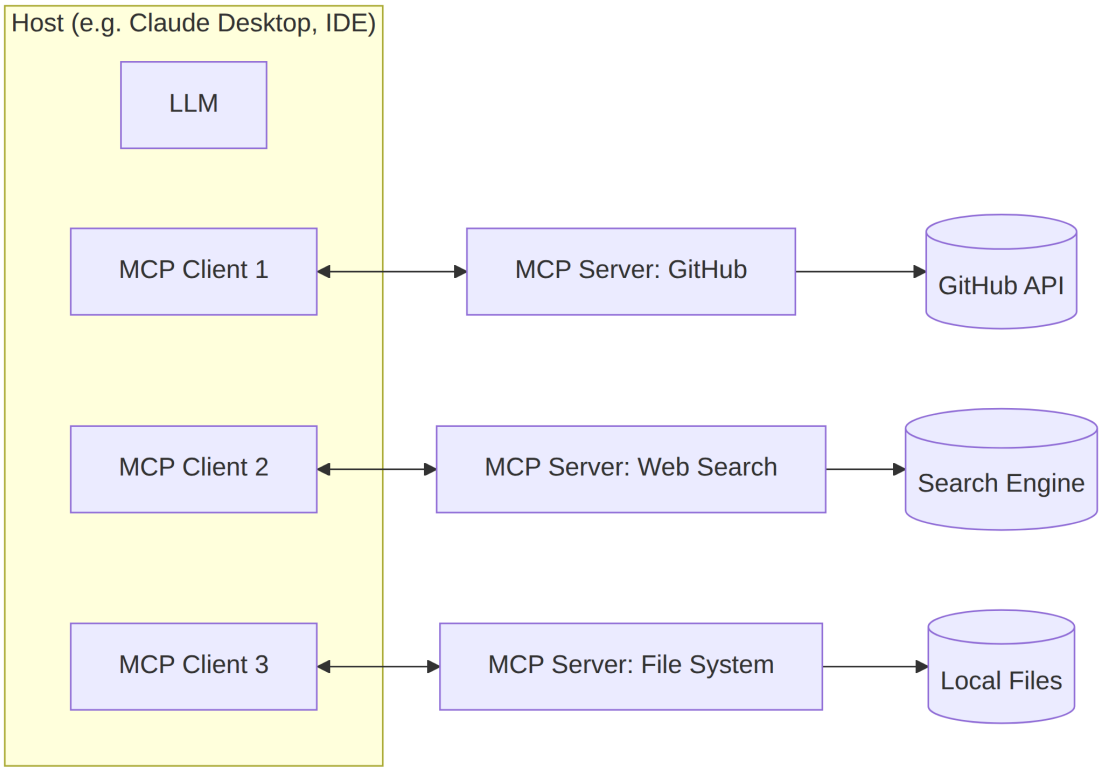
- **Hard to maintain:** API changes broke many integrations at once.

MCP addresses all four by standardizing the interface.

---

4. Part 2 — The What: MCP Architecture

4.1 The three roles



Role	What it is	Example
Host	The application the user interacts with. It runs the LLM and manages connections.	Claude Desktop, an IDE, a custom chat app
Client	A connector inside the host. Each client keeps a **one-to-one** connection with one server.	The component in Claude Desktop that talks to the GitHub server
Server	A lightweight program that exposes a tool or data source through MCP.	A GitHub server, a Postgres server, a web search server

Key point: one host can run many clients, and each client talks to exactly one server.

4.2 What servers can expose (server primitives)

Primitive	Controlled by	Purpose	Example
Tools	The model	Functions the LLM can call to take actions	`search_repos()`, `send_email()`
Resources	The application	Read-only data that provides context	A file, a database record, a document
Prompts	The user	Reusable prompt templates and workflows	"Summarize this PR", "Draft newsletter section"

4.3 What clients can offer servers (client features)

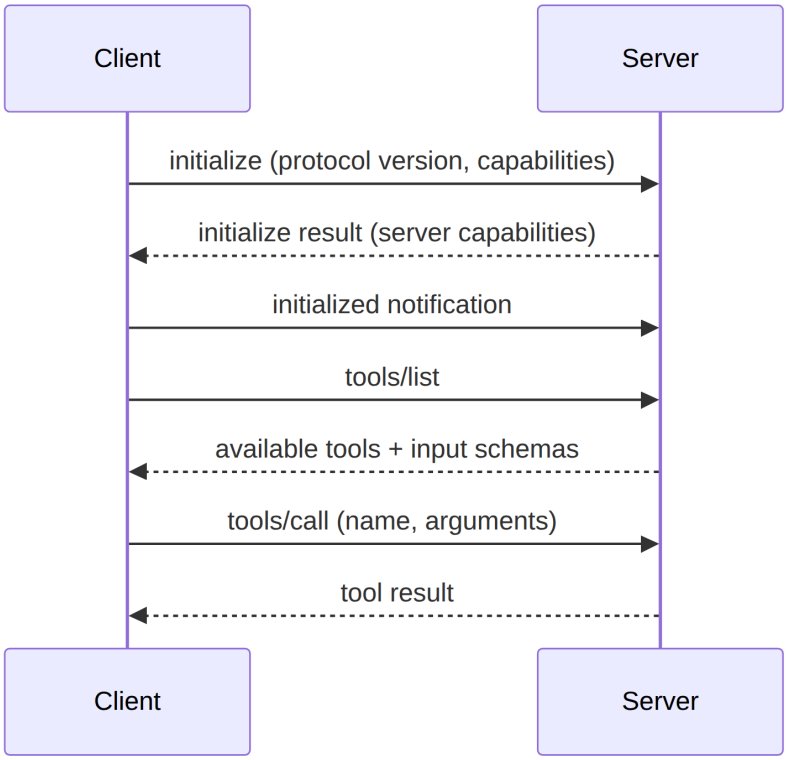
MCP is not one-way. Clients can also provide features to servers:

- **Sampling:** a server can ask the host's LLM to generate text, without needing its own model or API key.
- **Roots:** the client tells the server which folders or locations it is allowed to work in.
- **Elicitation:** a server can ask the user for extra input mid-task (for example, "Which repository did you mean?").

4.4 Communication layer

- **Message format:** JSON-RPC 2.0 (structured requests, responses, and notifications).
- **Transports:**
  - **stdio** — the server runs as a local process; best for local tools like file systems.
  - **Streamable HTTP** — the server runs remotely over HTTP; best for hosted or shared services. (The older HTTP + SSE transport has been replaced by this.)

4.5 Connection lifecycle



1. **Initialize:** client and server agree on a protocol version and share their capabilities.
2. **Discover:** the client asks which tools, resources, and prompts are available.
3. **Use:** the LLM decides which tool to call; the client sends the call; the server returns the result.
4. **Shutdown:** the connection closes cleanly.

---

5. Part 3 — The How: Building and Connecting MCP Servers

5.1 Connecting existing servers to Claude Desktop

Claude Desktop reads a JSON config file (`claude_desktop_config.json`). Adding a server is just adding an entry:

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/path/to/your/newsletter-folder"
      ]
    },
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "<your-token>"
      }
    }
  }
}
```

After restarting Claude Desktop, the tools from these servers become available to Claude automatically.

> ⚠️ Never commit real tokens to GitHub. Use environment variables or a secrets manager, and add config files with secrets to `.gitignore`.

5.2 Building a simple custom server in Python

Using the official Python SDK (`pip install "mcp[cli]"`), a minimal server looks like this:

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("newsletter-helper")

@mcp.tool()
def word_count(text: str) -> int:
    """Count the words in a newsletter draft."""
    return len(text.split())

@mcp.tool()
def reading_time(text: str, wpm: int = 200) -> str:
    """Estimate reading time for a draft."""
    minutes = max(1, round(len(text.split()) / wpm))
    return f"About {minutes} min read"

@mcp.resource("template://newsletter")
def newsletter_template() -> str:
    """The standard newsletter structure."""
    return "# Weekly AI Digest\n## Top Story\n## Tools\n## Research\n## Repo of the Week"

if __name__ == "__main__":
    mcp.run()  # uses stdio transport by default
```

The **docstrings and type hints matter**: they become the tool descriptions and input schemas the LLM reads to decide when and how to call each tool.

5.3 Testing your server

The **MCP Inspector** is a developer tool for testing servers before connecting them to a host:

```
mcp dev server.py
```

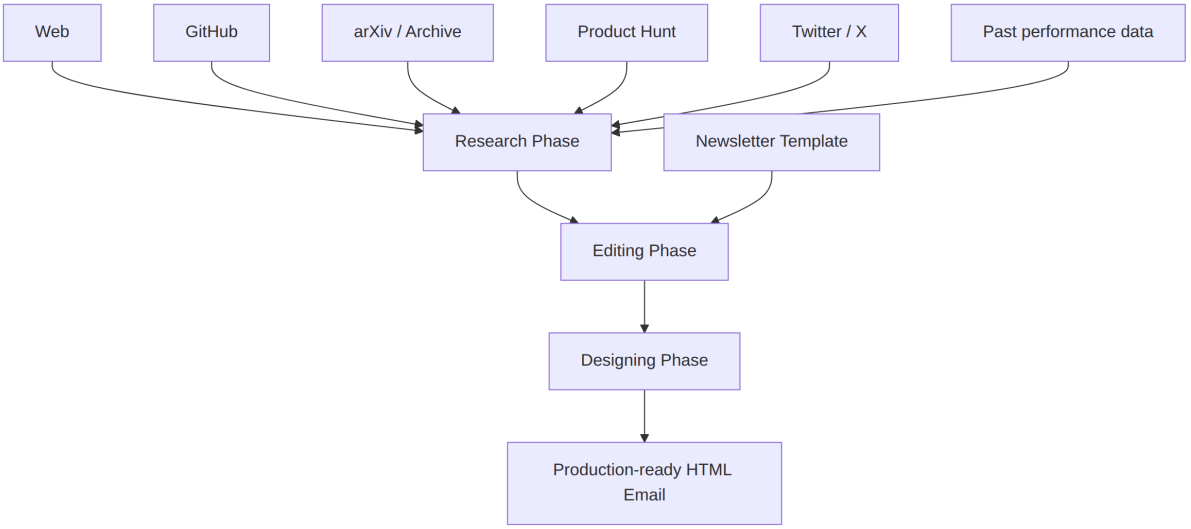
It lets you list tools, call them manually, and inspect responses.

---

6. Case Study: The AI-Powered Newsletter Project

Nitish automated the full lifecycle of a weekly AI newsletter using *Claude AI* connected to several tools through MCP (6:50–32:45).

6.1 Pipeline overview



6.2 Phase 1 — Research (13:30–16:58)

The agent gathers:

- **Content ideas** for the issue
- **Past performance data** (which topics or links got the most engagement)
- **Recent updates** from the web, GitHub, an archive source (likely arXiv-style research papers), Product Hunt, and Twitter

Why it matters: research is usually the most time-consuming part of a newsletter. With MCP, each source is just a server, and Claude decides which ones to query and how to combine results.

My added points:

- Using past performance data turns the newsletter into a feedback loop: content choices are guided by what readers actually engaged with.
- Pulling from several sources reduces the bias of relying on a single feed.
- It helps to have the agent record the source URL of every item so facts can be verified later.

6.3 Phase 2 — Editing (17:01–18:25)

The agent assembles the research into a **structured draft** following a provided template.

Why templates matter:

- They give every issue a consistent structure readers recognize.
- They constrain the LLM, which reduces rambling and keeps sections focused.
- They make the output predictable enough to hand over to the design step.

6.4 Phase 3 — Designing (18:30–19:55)

The draft is converted into a **production-ready HTML email template** with links and styling.

My added points on email HTML:

- Email clients are much stricter than browsers. Inline CSS and table-based layouts are the safest choice.
- Always test the output in several clients (Gmail, Outlook, mobile) before sending.
- Include a plain-text fallback for accessibility and deliverability.

6.5 Human-in-the-loop

Even with full automation, a human review step before publishing is recommended to check facts, tone, and links. Automation saves time on assembly; editorial judgment still matters.

---

7. Why "Minimal Coding" Works

Nitish emphasizes that the integration needed very little code. Most of the work was **providing configuration for MCP servers to Claude Desktop** (33:30–35:30).

Traditional approach	MCP approach
Write API client code for each service	Install an existing MCP server
Handle auth, pagination, errors per API	Server handles it internally
Write orchestration logic by hand	The LLM decides which tools to call
Rewrite everything to switch AI models	Any MCP-compatible host can reuse the servers

Why this is powerful: the community has already built many MCP servers for common tools. Once those exist, building an agent often becomes a matter of **choosing servers, writing good instructions, and providing templates**, not writing integration code.

---

8. Additional Insights and Best Practices

1. **Write clear tool descriptions.** The LLM picks tools based on names and descriptions. Vague descriptions lead to wrong tool calls.
2. **Keep tools focused.** One tool should do one thing well. `search_github_repos` is better than a generic `do_github_stuff`.
3. **Return structured, concise results.** Huge tool outputs waste the context window and confuse the model.
4. **Give the agent a clear workflow.** A system prompt or project instruction that says "Research → Edit → Design" works better than "make a newsletter."
5. **Use resources for templates and style guides.** Store the newsletter template and brand guidelines as MCP resources so they are always available.
6. **Log what the agent does.** Keep a record of which tools were called and what they returned, for debugging and auditing.
7. **Start with read-only tools.** Add tools that write, send, or delete only after the workflow is reliable.
8. **Version your configs.** Keep server configs (without secrets) in Git so the setup is reproducible.

9. Limitations and Things to Watch Out For

- **Security of third-party servers:** an MCP server runs code on your machine or accesses your accounts. Only install servers from trusted sources and review their permissions.
- **Prompt injection:** content fetched from the web (a tweet, a web page) may contain hidden instructions aimed at the AI. Treat fetched content as data, and keep a human review step before anything is published or sent.
- **Credential management:** tokens in config files can leak. Use least-privilege tokens (read-only where possible).
- **Hallucinated facts:** the LLM can still misread or invent details from research results. Verify claims and links before publishing.
- **Context window limits:** pulling too much data from many sources can exceed what the model can handle at once. Summarize per source before combining.
- **Rate limits and costs:** APIs such as GitHub or Twitter have rate limits; heavy automation can hit them.
- **Evolving standard:** MCP is still developing, so transports, SDKs, and features can change. Check the official spec for the latest version.

10. Ideas for Your Own MCP Projects

Project	MCP servers involved	What it automates
Research paper digest	arXiv/web search + file system	Weekly summaries of new papers in your field
Survey data assistant	PostgreSQL + file system	Natural-language queries over survey data
Job application helper	Web search + file system + email	Finding roles and drafting tailored cover letters
GitHub repo reporter	GitHub + file system	Weekly summary of issues, PRs, and commits
Data quality checker	Database + custom validation server	Running validation rules and reporting issues
Personal knowledge base	File system + local vector store	Q&A over your own notes and documents

11. Glossary

Term	Meaning
MCP	Model Context Protocol, an open standard for connecting AI apps to tools and data
Host	The AI application the user interacts with
Client	The connector inside a host that talks to one server
Server	A program exposing tools, resources, or prompts via MCP
Tool	A function the LLM can call to take an action
Resource	Read-only data provided as context
Prompt	A reusable prompt template exposed by a server
JSON-RPC 2.0	The message format MCP uses
stdio	Transport where the server runs as a local process
Streamable HTTP	Transport for remote servers over HTTP
Sampling	A server asking the host's LLM to generate text
Agent	An LLM that plans and calls tools to complete a multi-step task

12. Resources

- Official MCP documentation and specification: https://modelcontextprotocol.io
- Official MCP GitHub organization (SDKs and reference servers): https://github.com/modelcontextprotocol
- Python SDK: https://github.com/modelcontextprotocol/python-sdk
- Anthropic's announcement of MCP: https://www.anthropic.com/news/model-context-protocol
- CampusX YouTube channel (The MCP Trilogy series by Nitish)

Credits

Original content and newsletter project by **Nitish (CampusX)**. These notes add explanations, diagrams, code samples, and practical insights for learning purposes.